Snowflake-on-Helios 设计文档

在 Helios（FPGA 分布式 Union-Find 解码器）代码基础上实现 Snowflake（流式解码器）的改进方案

项	内容
文档状态	讨论稿 v0.1（不含实现代码）
参考论文 1	Namitha Liyanage, Yue Wu, Siona Tagare, Lin Zhong. *FPGA-based Distributed Union-Find Decoder for Surface Codes*, IEEE TQE 2024（工作目录：`FPGA-based Distributed Union-Find Decoder for Surface Codes.pdf`）
参考论文 2	Tim Chan. *Snowflake: A Distributed Streaming Decoder*, Quantum 2026（工作目录：`snowflake.pdf`）
参考代码	`Helios_scalable_QEC-main/`（GitHub: yale-paragon/Helios_scalable_QEC）
Snowflake 参考实现	Snowflake 论文的仿真器开源在 github.com/timchan0/localuf

---

目录

1. 背景与目标
2. 术语表：两篇论文中的名词解释
3. Helios 代码目录与文件作用
4. 两个算法的本质差异
5. 改进设计：模块级改动清单
6. 关键设计决策
7. 分阶段实施路线
8. 预期收益与代价
9. 验证计划与风险
10. 附录 A：关键数据速查
11. 附录 B：Snowflake 算法速查

---

1. 背景与目标

1.1 现状

- **Helios** 是目前最快的表面码 FPGA 解码器之一：在 VCU129 上实现了分布式 UF 解码器，d=21（唯象噪声）平均 11.5ns/轮，d=17（电路级噪声）平均 23.7ns/轮；借助上下文切换（Helios-n）可在同板解码 d=51。它支持加权边、擦除错误、滑动窗口（2d 高窗口批量解码 + 提交旧 d 层）三种扩展。
- **Snowflake** 是 UF 家族的最新发展：把 UF 改造成流式（streaming）解码器。相对 UF 精度提升约 25%（逻辑比特寿命 +24.9(5)%），平均运行时间标度从 UF 的略高于立方降为亚二次方；窗口高度从 2d 减为约 d（处理器数量减半）；核心创新是 **frugal 方法**（窗口逐轮上移、计算零丢弃）与 **2:1 簇生长调度**（half-edge 生长 + whole/half 簇记账）。

1.2 目标

在 Helios 现有 RTL 基础上实现 Snowflake 算法，期望获得：

1. **精度**：比 UF（即 Helios 现行算法）高约 25% 的逻辑比特寿命；
2. **面积**：PE 阵列减半 + 删除 direct-channel 消息网络；
3. **流式语义**：逐轮提交纠错（correction），天然适配实时 QEC；
4. **时钟频率**：所有通信退化为邻居链路，缓解 Helios 3D→2D 布线瓶颈。

1.3 总体结论

可行，且契合度高。 Snowflake 的分布式实现沿用 Macar/Actis 的"一节点一处理器 + 一边一链路"架构，与 Helios 同源（Snowflake 论文将 Helios 列为第一个本地化 UF 实现）。改动集中在 PE 数据通路与控制器状态机；Helios 的 PE 阵列组织、邻居链路、全局同步、控制树、上下文切换、多 FPGA 分区机制均可复用。

---

2. 术语表

2.1 QEC 与表面码基础

术语	英文	解释
量子纠错	QEC (Quantum Error Correction)	用冗余物理比特保护逻辑信息，同时避免"不可克隆"与"测量扰动"的限制。
表面码	surface code	拓扑纠错码，物理比特排成 2D 网格，只需近邻连通。旋转表面码（rotated）用 $2d^2-1$ 个物理比特编码 1 个逻辑比特，d 为码距。
码距	code distance, d	引发逻辑错误所需的最少物理比特翻转数。d 越大，逻辑错误率指数级下降，但需要更大的解码器。
数据比特 / 辅助比特	data qubit / ancilla qubit	数据比特承载编码信息；辅助比特（X 型/Z 型）与相邻数据比特纠缠，被周期性测量以探测错误。
稳定子测量	stabilizer measurement	对辅助比特的测量。一次测量即一轮（measurement round）。
综合征	syndrome	多轮测量结果的集合。解码器的输入。
缺陷	defect	某辅助比特本轮测量结果与上轮不同（探测事件被触发）。Helios 代码中对应 `is_error_syndrome`。
逻辑比特 / 逻辑错误	logical qubit / logical error	编码后的有效比特；解码错误导致其状态翻转即为逻辑错误。

2.2 噪声模型与解码图

术语	英文	解释

唯象噪声	phenomenological noise	只建模数据比特翻转（每轮前）与测量翻转；每个图顶点最多 6 条入射边。
电路级噪声	circuit-level noise	建模电路中任意门之间的错误；引入 hook error（辅助比特错误等效于相邻两个数据比特错误）；每个顶点最多 12 条入射边。Helios 论文的默认精度基准。
擦除错误	erasure error	可探测的比特泄漏。UF 解码器对擦除的处理是：擦除邻边权重置 0，先合并再生长（Helios 已实现，代价是多一个前置 merging 阶段）。
解码图	decoding graph	顶点 = 测量结果（探测事件），边 = 独立错误源，边权 = −log(错误概率)。把解码转化为图上的配对问题。Helios 图中唯象噪声下有 (d+1)·((d−1)/2)·d 量级顶点。
边界节点	boundary node / virtual boundary	虚拟顶点：只翻转一个探测器的错误机制用一条连到虚拟边界节点的边表示。Helios 代码中用"边界代价计数器"（`boundary_cost_*`）实现：簇生长耗尽代价即"碰到边界"。
西/东边界	west/east boundary	Z 存储实验解码图左右两侧的空间边界。一条从西到东贯穿的"值 1 边路径"构成逻辑翻转（Definition 5）。
未来边界	future boundary	流式解码中窗口顶部的虚拟边界：代表"尚未到来的测量结果"。forward 方法靠它处理跨越窗口边界的缺陷对；**Snowflake 不需要它**（见 §4）。
时间片 / 层	sheet / layer	sheet = 同一 t 坐标（同一测量轮）的所有顶点；layer = 解码图的周期性子图单位。

2.3 Union-Find 解码器（Helios 的算法基础）

术语	英文	解释
簇	cluster	图中相连的顶点集合。UF 从每个缺陷单点簇出发，交替生长与合并。
生长 / 生长值	growth / growth value	每条边维护的计数器，从 0 增长到边权 w。Helios 中每轮 +1；Snowflake 中每轮 +½（对应 0/½/1 三态）。
全生长边	fully grown edge	生长值 = 边权 的边。两个簇被全生长边连接即合并。Helios 代码：`is_fully_grown`。
奇簇 / 偶簇	odd/even cluster	缺陷计数为奇/偶的簇。UF 只生长奇簇（`is_odd_cluster`）。
根 / 根地址	root	簇的代表元素：簇内最小 ID 的顶点。Helios：`old_root`/`updated_root`。

并查集合并	UNION	合并时簇内所有顶点采纳最小 ID 为新 cid，并向传播来源设 parent，形成生成树。
生成树 / 剥离	spanning tree / peeling	偶簇内在生成树上配对缺陷、翻转路径上的边，得到纠错集。Helios 单 FPGA 版有专门的 peeling 阶段。
收敛广播	convergecast	沿生成树从叶到根汇聚信息（如奇偶性）。Helios 与 Snowflake 的 merging 阶段本质都是 flood + convergecast。
纠错 / 逻辑翻转	correction / logical flip	解码器输出：一组被翻转的边（消灭所有缺陷）；若纠错后仍有贯穿西东边界的"1 边路径"则为逻辑翻转。
逻辑错误率	logical error rate	每 d 轮测量中的逻辑翻转计数，衡量解码精度。

2.4 流式解码与 Snowflake

术语	英文	解释
批量解码	batch decoding	攒 d 轮综合征一次性解完。Helios 现行模式；实际系统中是流式问题的简化。
流式解码	stream decoding	解码图随时间不断增长，解码器须持续维护纠错。
前向方法	forward method	窗口（c+b 层）批量解码，提交 c 层、丢弃 b 层的计算。需 $b \gtrsim d$ 的窗口重叠以保证精度；每轮丢弃大量计算（功耗高）。
三明治方法	sandwich method	另一种窗口化流式解码方案（2002 年后文献中仅有的两种之一）。
frugal 方法	frugal method	**Snowflake 的流式框架**：窗口每轮只上移 1 层（c=1），缓冲区内所有计算状态保留复用——"零层计算被丢弃"，功耗减半。
解码窗口	decoding window	当前维护的子图 = 提交区 C（c 层）+ 缓冲区 B（b 层）。Snowflake：c=1，$b = b_{min} = 2\lfloor d/2 \rfloor \approx d$（Proposition 2），且无未来边界。
提交区 / 缓冲区	commit region / buffer region	窗口底部 c 层（其纠错可安全提交）/ 顶部 b 层（结果未定）。
暂定纠错 / 提交	tentative correction T / commit	解码器对窗口的输出 / 把 T 中提交区部分并入最终纠错 C。
缺陷推送 / 湮灭	push / annihilate	翻转边把缺陷沿边移到另一端；两个缺陷相遇互相湮灭。Snowflake 用缺陷的物理移动替代 Helios 的奇偶汇聚。

半条边生长	half-edge growth	Snowflake 每生长轮沿所有方向把边生长值 +½。
1:1 / 2:1 调度	1:1 / 2:1 schedule	每解码轮 1 次 / 2 次生长轮。1:1 下两个簇可能相距半条边而互相过度生长（违反 UF 的 Lemma 2），阈值仅 0.375%；2:1 先长 whole 簇再长 half 簇，阈值 0.735%，接近翻倍。
whole / half / mixed 簇	whole/half/mixed cluster	已生长偶数次 / 奇数次 的簇；whole 与 half 合并产物。2:1 调度下 mixed 簇不会出现（Proposition 1）。
CID	cluster ID	簇内最小节点 ID，该节点即根。节点沿全生长边看到更小 CID 即采纳并指向它。
指针	pointer	节点推送缺陷的方向（C/N/W/E/S/D/U 及对角组合，共 13 种）。沿指针走必达根。
洪泛 / 同步	flooding / syncing	merging 阶段的两个并行过程：flooding = CID/grown 沿全生长边传播；syncing = 缺陷沿指针逐边上推 + 活跃性从根回传。
去根化	unrooting	当指针指向窗口底部之外（簇随窗口上移而"掉出"），以 CID=reset 洪泛重置该簇所有节点的指针树，耗时 2 拍。Snowflake 特有。
drop / grow_whole / merging_whole / grow_half / merging_half	—	Snowflake 每解码轮的五个阶段（Fig.14）：drop 提交底行并下移一层（1 拍）；grow_whole（1 拍）；merging_whole（变长）；grow_half（1 拍）；merging_half（变长）。
时间步	timestep	Snowflake 分布实现的计算模型：一次同步的状态更新（近似对应硬件一拍）。
吞吐 / 时延	throughput / latency	每轮平均时间步数（要求 ≤ 测量周期）/ 最后一片数据到最终纠错提交的时长（对门隐形传态关键）。
泡利帧	Pauli frame	经典电子学记录的泡利算子偏移，纠错可只记账不物理施加。
Z 存储实验	Z-memory experiment	解码精度基准实验：持续解码 d 轮后做数据比特破坏性测量，统计 ^Z 逻辑可观测量的翻转。
阈值 / 梯度	threshold / gradient m	逻辑错误率 f = f_th·(p/p_th)^m 中的 (p_th, f_th) 与斜率 m。Snowflake 与 FM(UF) 的 m 几乎相同，精度差主要体现在阈值。

Macar / Actis	—	Snowflake 的直接架构前身（Actis 论文，Quantum 2023）：一节点一处理器、一边一链路、单控制器。

2.5 Helios 架构与 FPGA 术语

术语	英文	解释
PE	processing element	每个解码图顶点一个的处理单元，Helios 的并行基本单位（processing_unit.sv）。
控制器 / 控制节点树	controller / control node tree	全局阶段机。以 OR 树方式收集所有 PE 的 busy/odd 信号并广播阶段，避免单点瓶颈（decoder_stage_controller.sv + control_node_*.v）。
共享内存 S1–S4	shared memories	PE 间/PE-控制器间的相干共享状态：S1 = PE 本地状态（cid/odd/parent/st_odd，邻居可读）；S2 = 控制器可读状态（codd/stage/busy）；S3 = 边生长值（存于低 ID 端点）；S4 = global_stage（控制器写、PE 读）。
邻居链路	neighbor link	相邻 PE 间的直连线路，携带 fully_grown、root、increase、odd 信号（neighbor_link.sv）。
直接通道	direct channel	每个 PE 3 条带 FIFO 的阻塞通道 + 3D 距离求解器，把"奇偶/触界"消息按目标地址逐跳路由给新根（blocking_channel.sv、tree_distance_3d_solver.sv）。
全局阶段	global stage	控制器广播的阶段值：Helios 为 测量加载→(生长→合并→检查循环)→结果计算。
busy / 死锁检测	busy / deadlock	PE 自报未完成（busy），控制器等全部 busy=0 才换阶段；超时触发 deadlock。
上下文切换 / Helios-n	context switching	把解码图切成 n 个子图，n 个虚拟 PE 分时复用一个物理 PE（状态存 LUTRAM，1 拍换入换出）。Helios-51 即 51 个虚拟 PE 映射到 1 个物理 PE，用于 d=51。
晶格 / 子图划分	lattice / sub-graph partition	物理 PE 阵列（晶格）与解码图子图的一一映射；划分由 partitionScheme.py（pymetis）完成。
leaf / hub / 路由表	leaf FPGA / hub FPGA / routing table	多 FPGA 拓扑：leaf 放 PE 阵列，hub 转发消息，路由表（routing_table.sv）给出目的 FPGA→出口 FIFO 的映射。
仲裁 / 串并转换	arbitration / serdes	多条消息流合并进一条物理互连（pu_arbitration.sv、final_arbitration.sv）与宽窄 FIFO 转换（serdes.sv）。
互连模型	interconnection model	仿真中模拟跨 FPGA 链路固定延迟的模型（interconnection_model.sv）。

VCU129 / ZCU106	—	论文评测用 VCU129（Virtex UltraScale+ 56G PAM4 评估板）；仓库 README 声称支持 ZCU106。
LUT / reg / BRAM / LUTRAM / DSP	—	FPGA 资源。Helios 只用 LUT+寄存器（无 DSP），上下文切换用 LUTRAM，BRAM 仅用于 MicroBlaze 接口。
f_max	—	最大时钟频率。Helios 的 f_max 随 d 增大而下降，主因是 3D 结构映射到 2D 芯片的布线延迟（论文估计约 d≈1800 时撞墙）。

---

3. Helios 代码目录与文件作用

> *路径相对 `Helios_scalable_QEC-main/Helios_scalable_QEC-main/` 。仓库是**模板 + 生成脚本**结构：`generate.py` 用 `build_scripts/templates/` 下的模板生成 `design/generated/` （本仓库发布时不带该目录）。*

```
.
├── build_scripts/       # 代码生成与 Vivado 工程脚本
│   ├── generate.py              # 多 FPGA 版主生成脚本：按用户配置从模板生成 RTL（路由表、hub 顶层、总 testbench 等）
│   ├── single_FPGA_generate.py  # 单 FPGA 版生成脚本（对应单板实验流程）
│   ├── partitionScheme.py       # 用 pymetis 把 PE 阵列划分给各 leaf FPGA（含网格 IO/边界互联计算）
│   ├── user_configuration.py    # 用户配置：codeDistanceX/Z、控制树结构、互连位宽/延迟、随机错误生成开关
│   ├── tcl_script_hub_full_gpio.tcl / tcl_script_leaf_full_gpio.tcl  # hub/leaf FPGA 的 Vivado 工程脚本
│   └── templates/               # 带 /*$$KEY*/ 占位符的 RTL 模板
│       ├── routing_table.sv             # 路由表模板：目的 FPGA → 下行 FIFO 编号
│       ├── top_module_hub.sv / top_module_for_leaf*.sv  # hub/leaf 顶层模板
│       ├── hub_wrapper.sv / leaf_wrapper*.sv / root_hub*.v  # 各节点包装模板
│       ├── interconnection_model_wrapper*.sv  # 跨 FPGA 互连模型包装（含 ll_connected 变体）
│       ├── decoder_stage_controller_master.sv / _dummy.sv  # 总控制器/占位控制器模板
│       ├── standard_planar_code_2d.sv    # 2D 平面码 PE 阵列模板
│       └── rand_gen_top.sv              # 随机错误注入顶层模板
├── design/              # 通用 RTL（所有配置共用）
│   ├── pe/
│   │   ├── processing_unit.sv                  # ★ 可扩展版 PE：SPREAD_CLUSTER（并簇）/ GROW_BOUNDARY（生长+边界代价）/ SYNC_IS_ODD_CLUSTER（奇簇同步）/ MEASUREMENT_LOADING 四阶段；含 direct channel 收发、tree 比较求解器、边界代价计数
│   │   ├── processing_unit_single_FPGA_v2.v    # 单 FPGA 版 PE：EXPOSED_DATA 接口暴露 root/odd/parent 等给邻居；带 cluster parity、touching boundary、peeling（纠错剥离）逻辑
│   │   └── processing_unit synthesizable_test.sv  # 可综合测试变体
```

```
|   ├── channels/
|   |   ├── neighbor_link.sv                    # ★ 边链路：两端共享的生长计数器（LENGTH=2*边权）、fully_grown 判定、双方 root 转发、奇簇位同步
|   |   ├── neighbor_link_v2.v / neighbor_link_internal_v2.v / neigbor_link_fifo.sv  # 多 FPGA 版边链路（挂 hub FIFO）与内部实现
|   |   ├── blocking_channel.sv                 # 阻塞式握手通道（内嵌 fwft FIFO，作为 PE 内部流水级）
|   |   ├── nonblocking_channel.sv / nonoblockingchannel_fifo.sv  # 非阻塞通道（满则丢弃/旁路）及 FIFO
|   |   ├── pu_arbitration.sv                   # PE 列仲裁：把邻居链路消息与 direct 消息合并进每列的主 FIFO
|   |   ├── final_arbitration.sv / final_arbitration _ll_connected.sv  # hub 顶层最终仲裁（合并多路消息到物理互连；ll_connected = 低延迟直连版）
|   |   ├── serdes.sv                           # 宽内部 FIFO ↔ 窄物理互连的串并转换器
|   |   ├── interconnection_model.sv            # 跨 FPGA 链路固定延迟仿真模型（计数器实现）
|   |   ├── error_stream.sv / rand_generator.sv / simple_rand_setter.v  # 仿真用随机错误流生成（64 位 PRNG）
|   |   └── many_to_one_mux.sv                  # 多选一复用器
|   ├── generics/
|   |   ├── tree_compare_solver.sv              # ★ 树形最小值求解器：O(log n) 深度选出 CHANNEL_COUNT 路+默认值中的最小数（PE 里用来选"最小 root"）；同文件含 min_val_*_with_index 系列
|   |   ├── tree_distance_3d_solver.sv          # 3D 维度序路由：给定目标地址，按 Z→X→Y 顺序选出口通道（direct channel 的消息路由用；完整距离比较树已被注释掉）
|   |   ├── fifo_fwft.v                         # 先读先出（first-word-fall-through）FIFO
|   |   ├── ram.sv                              # 简单 RAM
|   |   └── many_to_one_mux.sv 的替代小工具（同目录复用）
|   ├── stage_controller/
|   |   ├── decoder_stage_controller.sv         # ★ 全局阶段机：GROW→SPREAD→SYNC 循环 + 死锁检测 + 迭代/周期计数 + 结果计算触发
|   |   ├── decoder_stage_controller_partial.sv # 阶段机变体（部分功能/调试用）
|   |   ├── control_node_single_FPGA.v          # 单 FPGA 控制器：收集 busy/odd、广播 global_stage
|   |   ├── control_node_child_FPGA.v / control_node_root_FPGA.v  # 多 FPGA 控制树的中继节点/根节点
|   |   └── get_boundry_cardinality_3d.sv       # 结果计算器：统计触碰边界的奇簇以输出逻辑翻转（当前被 stub 为 0，注释里保留完整实现）
|   └── wrappers/
|       ├── Helios_single_FPGA_core.v / Helios_single_FPGA_wrapper.v  # 单 FPGA 顶层核心/包装
|       ├── single_FPGA_decoding_graph.v        # ★ 批量解码图生成器：按 (i,j,k) 三维实例化 PE 阵列 + 邻居链路，输出 roots/odd/busy/correction（WEIGHT_X/Z/M 可配）
|       ├── single_FPGA_decoding_graph_dynamic.v / _rsc.sv  # 动态权重版 / 资源优化（rsc）版
```

```
│       ├── standard_planar_code_2d*.sv          # 2D 平面码 PE 阵列多个变体：基础版、left/right、no_fast_channel（无 direct channel）、with_stage_controller、synthesizable_top
│       ├── left_with_stage_controller.sv / right_with_stage_controller.sv  # 左右半边阵列 + 各自阶段控制器
│       ├── mem_communicator.v                   # 片上存储通信（BRAM 接口）
│       └── qec_3_axi_v1_0.v / qec_3_axi_v1_0_S00_AXI.v  # AXI4-Lite 从机封装：MicroBlaze 经 AXI 读写综合征/结果
├── parameters/
│   └── parameters.sv        # 共享参数：STAGE_WIDTH 与各阶段常量（当前定义与 scalable 版代码不完全一致，见 §3.1）
├── test_benches/
│   ├── unit_tests/          # 各模块单元测试（blocking/nonblocking channel、neighbor link、serdes、tree solver、processing unit、stage controller 等）
│   ├── full_tests/          # 整机测试：axi_test_bench.v、bench_decoder.sv、bench_multi_fpga.sv、bench_resource_usage.sv、bench_usual_latency.sv、single_FPGA_verification/latency/FIFO 测试等
│   └── test_data/           # d=3/5/7/9/11/13 的输入综合征与期望输出（含 rsc 与 deadlock 用例）
├── plots/                   # gnuplot 画图脚本（论文图 8–12 的复现）
├── scripts/                 # 实验脚本：ave_time_calc.py（平均时延统计）、linear_noise.py / linear_weight.py（噪声/权重配置）、comp_to_linear.py、simple_eval.py、project_vu19_test.tcl、total.tcl（建 Vivado 工程）
├── old_files/               # 历史版本（早期 pe.v、router.v、top_single_3/5/7_by_7_x.v、早期 Vivado 工程与参数文件），已废弃
├── README.md                # 仓库说明（指向 arXiv:2301.08419 早期版本论文）
└── LICENSE
```

3.1 已知构建问题（动手前必须先修）

1. **阶段常量不一致**：`parameters/parameters.sv` 定义的是 `STAGE_GROW / STAGE_MERGE / STAGE_PEELING / ...`（单 FPGA 版用），而 scalable 版代码（`processing_unit.sv`、`decoder_stage_controller.sv`、模板）使用的是 `STAGE_SPREAD_CLUSTER / STAGE_GROW_BOUNDARY / STAGE_SYNC_IS_ODD_CLUSTER` ——**后者在整个仓库中没有任何定义**（大概率由构建流程在 `design/generated/` 中生成，但该目录未随仓库发布）。
2. `design/generated/` 缺失：需先运行 `python generate.py`（依赖 numpy、pymetis）确认生成产物与模板占位符匹配。
3. README 声明支持 ZCU106，论文评测用 VCU129：板级验证目标需先明确。

---

4. 两个算法的本质差异

维度	Helios（现有代码）	Snowflake	影响
解码方式	批量：d 轮 3D 图一次解完；滑动窗口 = 2d 高窗口批量解、提交旧 d 层	流式（frugal）：窗口高约 d（c=1, b=b_min=2⌊d/2⌋），每轮下移 1 层、提交 1 层，计算全部复用	控制器循环 + 数据

			流重构
未来边界	有（UD 边界，k=0 层，cost 计数器）	无（簇下落速度是生长 2 倍，永不触及）	删掉 UD 逻辑，省面积
生长粒度	整数步长 +1/轮	半条边 +½/生长轮，且 2:1 调度（whole 先、half 后）	边计数器加 1 位；生长触发拆两次
簇奇偶性	direct channel 消息路由到根做 XOR 汇聚	缺陷沿指针逐边上推，相遇湮灭；移动即翻边	用局部移动替代消息网络
簇记账	cid/parent/odd/st_odd + 阶段同步	增加 whole/half/grown/pointer/unrooted/busy	每 PE 加少量寄存器
根掉落处理	无（批量图固定）	unrooting：指针树就地重建	新机制，约 2 拍/事件
纠错输出	解码结束后统一算（peeling / 结果计算器）	逐轮：底行 owned 边的 correction 位随 drop 移出	输出路径改为流式
控制器	3 阶段循环 + busy OR + 死锁检测	5 阶段循环 + busy OR + "无缺陷只跑 drop"快速通道	状态机重写，控制树复用

精度	= UF	比 UF 高 24.9(5)% 寿命（阈值达 FM(UF) 的 ~97%）	本次改进的主要收益
每轮时延（算法标度）	批量平均时延随 d 亚线性（FPGA 实测）	时间步数随 d 亚二次方（仿真器），比 FM(Macar) 慢 0.9–3.5×	吞吐换精度+面积

---

5. 改进设计：模块级改动清单

5.1 `processing_unit.sv`（PE，核心改动）

改动	内容
新增寄存器	`defect`（缺陷粒子位）、`pointer`（4 位方向编码：C/N/W/E/S/D/U + 6 个对角）、`whole`、`grown`（本轮首生长轮是否已长）、`unrooted`、`busy`
复用/改名	`updated_root` → CID；`is_odd_cluster` → active；`is_odd_cardinality` → 根处缺陷奇偶（湮灭 = XOR）
生长逻辑	`neighbor_link` 的 growth 计数器改为 0/1/2 三态（= 0/½/1）；`STAGE_GROW_BOUNDARY` 单触发拆为 grow_whole（条件 `active && whole`）与 grow_half（条件 `active && !whole && !grown`）两次
删除	direct channel 收发逻辑、`tree_distance_3d_solver` 实例、UD 边界计数器与比较器（`boundary_ud`/`boundary_increased_ud`）
边界处理	保留 `has_boundary_z`/`boundary_cost_z` 成本计数器实现西/东边界；触界即失活；加**西向优先**确定性判决（附录 E：恒推西边界比随机选略优）
缺陷移动	在 merging 阶段按 pointer 方向把 defect 推给对应邻居（沿 `neighbor_link` 已有的边信号加一位）；移动同时翻转该边的 correction 位（存于低 ID 端点的 owned 边寄存器）
无缺陷快速通道	参与全局"窗口内是否有缺陷"OR（供控制器做 drop-only 判定）

5.2 `neighbor_link.sv`（小幅扩展）

- 保留：growth 计数器、`is_fully_grown`、双方 root 转发、奇簇同步。
- 新增：双向 `pointer` 方向位、`grown` 标志位、`defect` 推送位（各 1–4 bit）。
- growth 计数器增量语义从"+1"改为"+½"（三态编码）。

5.3 `decoder_stage_controller.sv`（重写状态机）

- 五阶段循环：`drop`(1 拍) → `grow_whole`(1 拍) → `merging_whole`(变长，等 busy 清空 + wait 1) → `grow_half`(1 拍) → `merging_half`(变长，等 busy 清空 + wait 2)，对应 Snowflake Algorithm 6。
- 保留 busy 全局 OR、`DEADLOCK_THRESHOLD` 死锁检测、迭代/周期计数器。
- 新增 1 bit 全局 OR：窗口内是否有缺陷（无缺陷 → 只循环 drop）。
- 去掉 `STAGE_MEASUREMENT_LOADING` 的**全量状态重置**（流式下状态跨轮保留；改为仅顶行注入新测量差分）。

5.4 drop 的实现（关键决策，见 §6.1）

- **方案 A（推荐）**：沿 K 轴移位寄存器。每个 PE 在 drop 拍把相对变量（CID/defect/pointer/whole/grown/correction…）发给下层 PE、接收上层 PE 的，经现有垂直邻居链路完成。静态 ID 语义不变（顶层 ID 最小），unrooting 逻辑原样照搬。
- **方案 B（备选）**：层映射旋转（ring pointer），物理 PE 不动、逻辑层号轮转。省布线但破坏静态地址假设，unrooting 语义需重构。

5.5 提交路径

- correction 位随缺陷移动即时翻转；drop 时把最底层 PE 的 owned 边 correction 串行移出 → 逐轮纠错输出流。替代 Helios 的批量结果计算路径（`get_boundry_cardinality_3d` 可退役或改为统计逻辑翻转的流式累加器）。

5.6 多 FPGA 分区（`partitionScheme.py` / hub-leaf 复用）

- Snowflake 垂直操作天然串行，垂直方向仅"下坠一层"一条边界流量 → **沿 K 轴切分**最自然：每个 leaf 管若干连续层，跨界只需最底/顶两层间的邻居链路。
- direct channel 退役后跨 FPGA 流量比 Helios 现状更小；hub 树 + `routing_table` + `final_arbitration` 复用。

5.7 直接复用的 Helios 资产

- **加权边**：Snowflake 的"半条边"与 Helios 整数权重天然兼容——权重刻度 ×2（`neighbor_link` 的 LENGTH=2·w 已有该语义）。Snowflake 论文 future work 点名的加权生长，Helios 已实现。
- **上下文切换（Helios-n）**：Snowflake 窗口高度减半后，同样 d 所需 PE 更少；叠加 context switching 可进一步换面积/增大 d。
- **控制树 / 全局同步 / 死锁检测 / serdes / 互连模型 / AXI 主机接口 / 随机错误注入**：全部不变。

---

6. 关键设计决策

#	决策点	选项	建议
1	drop 的实现	A：垂直移位寄存器；B：层映射旋转	**A**（正确性易对照仿真器；B 作为二阶段优化）
2	direct channel 取舍	保留（沿用 Helios 消息网络） / 退役（纯局部指针移动）	**退役**：省面积、短关键路径、跨 FPGA 流量更小；代价是缺陷移动逐边多拍（Snowflake 时间步模型本就有此语义）
3	边界表示	成本计数器（Helios 原生） / 显式边界 PE（Snowflake 原生）	**成本计数器** + 西向优先平局裁决（改动最小）

4	顶点 ID 约定	Helios：k=0 顶层 ID 最小，根在顶层；Snowflake：高层节点 ID 小（Remark 7：边界节点 ID < 探测器）	沿用 Helios 地址布局即可，注意确认"根在窗口高层"的等价性；必要时做 ID 重映射宏
5	多 FPGA 切分轴	X/Y/K 任意 / 仅 K	**仅 K**（匹配 drop 的单向数据流）
6	目标平台	ZCU106（README）/ VCU129（论文）	先按论文配置复现基线，平台迁移放后
7	2D 压平架构	Snowflake 论文 future work（每 (x,y) 列一个处理器、垂直操作串行）	作为**二期目标**：对 FPGA 布线友好，但改动量大，不与一期混做

---

7. 分阶段实施路线

阶段	内容	验收标准
0 基础设施	修复构建：补 STAGE 常量定义、跑通 `generate.py`、生成 `design/generated/`；在 d=3/5 复现 Helios 仿真与综合基线	现有测试全部通过，拿到对照数据
1 黄金向量	移植 Snowflake 仿真器（timchan0/localuf）输出 RTL testbench 输入/期望输出；小 d（3–9）对齐	RTL 与仿真器逐拍一致
2 批内改动（最小风险切入）	保持批量解码，仅换生长规则：half-edge + 2:1 调度 + whole/half/grown；边界西向优先	精度收益复现（逻辑错误率下降 ~25% 量级）；时延增量可接受
3 流式化	drop 垂直下移（方案 A）、删除未来边界与 direct channel、correction 流式提交、unrooting	持续流式解码正确；窗口高度 ≈ d
4 控制器重写	五阶段状态机 + busy 等待 + 无缺陷 drop 快速通道	各阶段时长与仿真器一致
5 优化	加权边（×2 刻度）、b_min 可配、context switching、沿 K 轴多 FPGA 分区	资源/时延权衡曲线；跨板仿真通过
6 评测	d=9/13/17，唯象 + 电路级噪声；逻辑错误率、逐轮时延分布、LUT/功耗、f_max 四组数据 vs Helios 基线	设计文档更新为实测报告

---

8. 预期收益与代价

收益

1. **精度**：比 UF 高 24.9(5)% 逻辑比特寿命（阈值达 FM(UF) 的 ~97%）。Helios 现行滑动窗口即 FM(UF)，收益直接成立。
2. **面积**：窗口高度 d vs 2d → PE 阵列减半；删除 direct channel FIFO、3D 距离求解器、UD 边界逻辑后单 PE 面积进一步下降。

3. **流式语义**：逐轮提交、无 2d 窗口缓冲、计算零丢弃（论文称功耗减半，对低温环境片上解码有意义）。
4. **时钟频率**：全部通信退化为邻居链路，缓解 Helios 的 3D→2D 布线瓶颈（其论文估计 d≈1800 撞墙）。

代价（诚实预期）

1. **每轮时延**：Snowflake 仿真器为 FM(Macar) 的 0.9–3.5 倍（$p=10^{-3}$）。FPGA 全局同步下 merging 洪泛可单拍完成（Snowflake 论文自注：Syncing/Flooding 换序即可加速，仿真用的是保守顺序），差距会缩小，但**逐轮时延大概率仍高于批量 Helios**——吞吐换精度+面积+流式的明确权衡。
2. unrooting 与 2:1 调度的边角情况多，验证工作量是最大风险。
3. 仓库构建问题（§3.1）需先投入修复。

---

9. 验证计划与风险

9.1 验证重点（按风险排序）

1. **2:1 调度正确性**：专门构造"两个簇相距半条边"用例（Lemma 2 被违反的场景）；断言 mixed 簇永不出现（Proposition 1）。
2. **unrooting**：构造指针指向窗口底之外的用例；验证 2 拍完成、不反向传播。
3. **drop 数据一致性**：连续多轮移位后 CID/pointer/correction 与仿真器逐拍比对。
4. **边界平局**：簇同时触及西/东边界的用例，确认恒推西边界。
5. **死锁**：长链错误 + 高噪声下的死锁检测不误触发。

9.2 主要风险

风险	缓解
RTL 与仿真器语义偏差（尤其 pointer 方向编码 13 态）	阶段 1 的黄金向量 + 逐拍比对
每轮时延劣化超过预期	阶段 2 单独测量"仅换生长规则"的时延增量，与流式化成本分离
f_max 不升反降（新增移位链路）	移位走已有时钟域寄存器流水；必要时回退方案 B
多 FPGA 分区引入新问题	一期只做单 FPGA；分区留到阶段 5 且仅沿 K 轴

附录 A：关键数据速查

A.1 Helios（IEEE TQE 2024）

指标	数值
d=21 唯象 p=0.001 平均时延	11.5ns/轮（100MHz）
d=17 电路级 p=0.001	23.7ns/轮（75MHz）
d=13 唯象 p=0.001	15ns/轮

d=51（Helios-51，资源优先）	543.9ns/轮 @p=0.001；1064ns @p=0.005；~796K LUT；85MHz
滑动窗口 d=13	371K LUT（vs 166K 有限窗口）；194→230ns
资源	d=21 唯象 898,715 LUT / 238,939 reg；电路级约 2×
功耗	d=13 全活跃估计 4.639W（实际解码更低）
最坏情况	$3d^4+2d$ 拍；d=13 理论 857μs，10^6 实测最长 920ns
阈值（UF）	唯象 0.024；电路级 0.0078（Barber et al.）

A.2 Snowflake（Quantum 2026）

指标	数值
阈值 (p,f)_th	FM(UF) $(0.755,10.5)\times10^{-2}$；Snowflake 2:1 $(0.735,6.8)\times10^{-2}$；1:1 $(0.375,9)\times10^{-2}$
寿命提升	+24.9(5)%
平均运行时间标度 m（T∝d^m）	FM(Macar) 亚线性：0.69–0.94；Snowflake 亚二次：1.16–1.77（$p=10^{-4}$~4×10^{-3}）
相对速度	0.9–3.5× 慢于 FM(Macar)（$p=10^{-3}$），但处理器数不足其一半
窗口	c=1，b=b_min=2⌊d/2⌋，无未来边界
每解码轮阶段	drop(1) → grow_whole(1) → merging_whole(变长) → grow_half(1) → merging_half(变长)

附录 B：Snowflake 算法速查

B.1 每节点变量（Appendix D.1）

相对变量：`active`、`whole`、`CID`（reset 表示待去根）、`defect`、`pointer`（C/N/W/E/S/D/U/NU/WD/EU/SD/NWD/SEU）；绝对变量：`grown`、`unrooted`、`busy`、`stage`。
每边变量：`growth` $\in \{0,½,1\}$、`correction` $\in \{0,1\}$，存于低 ID 端点（owned edges）。

B.2 每解码轮（Algorithm 5，2:1 调度）

1. raise W by 1 layer（drop：提交底行 correction，相对变量下移，顶行注入新缺陷）；
2. 对每个 active whole 簇：Grow（边 growth +½，翻转 whole）；
3. 每个簇 Merge（定根：触界则根=边界，否则取最高 t 节点；缺陷沿路径推向根、翻转路径边）；
4. 对每个 active half 簇（且本轮未长）：Grow；
5. 每个簇 Merge。

B.3 关键性质

- Lemma 1：簇失活 ⇔ 缺陷数偶 或 触界。
- Lemma 2（UF 定理）：簇生长时被覆盖的初始"1 边"数 +½ → 任意 s<d/2 个错误可纠正。

- Proposition 1：2:1 调度下 mixed 簇永不出现。
- Proposition 2：保证 Condition 1 的最小缓冲区高度 b_min = 2⌊d/2⌋。
- Remark 7：边界节点 ID 小于探测器 → 根只需检查"自己是否是边界节点"即可判断触界。

文档由会话讨论整理生成；后续阶段产出（构建修复、黄金向量、RTL 改动）可另开文档跟进。